

CBIS-DDSM y pipeline interpretable

ResNet-18, KAN y flujo de notebooks

Tesis — Interpretabilidad en mamografías

29 de abril de 2026

- 1 Base de datos: CBIS-DDSM
- 2 Pipeline notebooks/ — una vista por notebook
- 3 Experimento Prueba_embeddings/
- 4 Mini-proyecto Prueba_red_KAN
- 5 Cierre

Las métricas que siguen provienen de las **salidas guardadas** en los notebooks del repositorio (última ejecución registrada). Si vuelves a entrenar con otra semilla, hardware o datos, los valores pueden cambiar; la interpretación cualitativa del flujo se mantiene.

- Subconjunto **curado** del DDSM: mamografías para investigación en detección y diagnóstico asistido.
- En este proyecto se usan:
 - **CSV** de descripción de casos (masas y calcificaciones) y metadatos DICOM.
 - **Imágenes JPEG** de mamografías completas alineadas con esas tablas.
- Tarea: clasificación **binaria** BENIGNO / MALIGNO a partir de imágenes de **mamografía completa** (vista acordada con el manifiesto).

- `src/data/raw/csv/` — tablas oficiales (`calc_*`, `mass_*`, `dicom_info.csv`, etc.).
- `src/data/raw/jpeg/` — imágenes descargadas.
- `scripts/build_resnet18_manifest.py` → `manifest_{train,test,all}.csv` en `src/data/processed/`.
- Split por `patient_id` en `train/val` para limitar fugas; hold-out de test fijado por el manifiesto.

Ejemplos de filas en tablas crudas (según ejecución del notebook):

- `calc_case_description_test_set.csv`: ~ 326 filas $\times$ 14 columnas.
- `mass_case_description_test_set.csv`: ~ 378 filas (otra ejecución muestra ~ 1318 en conjunto train).
- `dicom_info.csv`: del orden de 10^4 filas (metadatos por series/imagen).

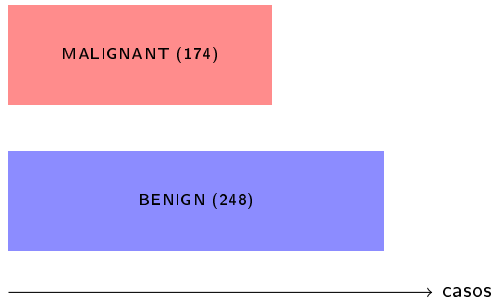
Tras el manifiesto, el conjunto de **test** usado en los modelos es de **422** casos: 248 BENIGN y 174 MALIGNANT (proporción coherente en varios notebooks).

Distribución de clases en test (422 casos)

Clase	Casos	Proporción
BENIGN	248	58,8 %
MALIGNANT	174	41,2 %

Lectura:

- Hay desbalance moderado a favor de BENIGN.
- Accuracy sola no basta: conviene mirar recall de MALIGNANT.



Objetivo: inspección exploratoria de CSV (patología, vistas, rutas a ficheros).

- Carga de `calc_*` y `mass_*` en train/test y metadatos DICOM.
- Comprobación de columnas clave: `pathology`, rutas de imagen, identificadores de paciente.
- Base para confiar en que el script de manifiesto y los loaders ven datos alineados.

Setup: ResNet-18, cabeza binaria, mismo split test que el resto del proyecto (422 imágenes).

Resultados (test, umbral 0,5 en argmax)

	BENIGN	MALIGNANT
Precision	0,66	0,50
Recall	0,60	0,56

Accuracy global $\approx$ **0,59**. Matriz de confusión (filas = verdadero): $\begin{pmatrix} 149 & 99 \\ 76 & 98 \end{pmatrix}$.

Punto de partida antes de fine-tuning agresivo ni KAN.

Objetivo: fine-tuning en dos fases (cabeza $\rightarrow$ backbone+cabeza), pesos de clase, early stopping por AUC en validación.

- Mejor **AUC en validación: 0,7168** (época destacada en P2).
- **Evaluación en test** (mejor checkpoint según val):
 - AUC = **0,6989**, accuracy $\approx$ **0,642**, recall MALIGNANT $\approx$ **0,615**.
 - Matriz de confusión: $\begin{pmatrix} 164 & 84 \\ 67 & 107 \end{pmatrix}$.
- Checkpoint guardado p.ej. `resnet18_tuned_best.pt` para embeddings y etapas posteriores.

Objetivo: elegir umbral en probabilidad de MALIGNO distinto de 0,5 según restricción de recall clínico.

- **Test AUC** del modelo calibrado en score: **0,6989** (coherente con el notebook 02).
- Umbral seleccionado (ejemplo de salida): **0,3216**.
- A ese umbral: precisión MALIGNANT $\approx$ **0,53**, recall MALIGNANT $\approx$ **0,80**, $F_1 \approx$ **0,64**.
- Comparación con 0,5: mayor recall de malignos a costa de precisión; configuración exportada a JSON bajo `reports/models/`.

Impacto del umbral en MALIGNANT

Umbral	Accuracy	Precision mal.	Recall mal.	F_1 mal.
0,50	0,642	0,560	0,615	0,586
0,3216	0,626	0,530	0,805	0,639

Cómo interpretar la tabla:

- Al bajar el umbral, sube el recall de malignos (menos falsos negativos).
- Se sacrifica algo de precisión y accuracy global, pero mejora el F_1 de MALIGNANT.
- Para un escenario clínico de cribado, esta dirección suele ser preferible.

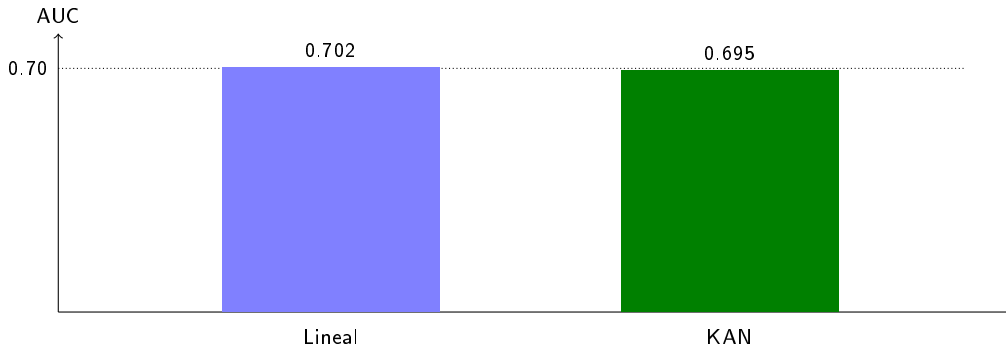
Embeddings 512-D del ResNet afinado; cabeza lineal y KAN con capas $512 \rightarrow 16 \rightarrow 2$ entrenadas con early stopping por AUC en val.

Test (última corrida registrada)

Cabeza	Acc	AUC	Rec. mal.	F_1 mal.
Lineal	0,604	0,702	0,851	0,639
KAN	0,609	0,695	0,845	0,641

En esta ejecución el lineal ganó ligeramente en AUC; la KAN prioriza forma funcional interpretable frente a solo ganancia puntual de AUC.

Comparativa visual de AUC (test)



Lectura: en esta corrida, lineal gana levemente en AUC; KAN mantiene desempeño cercano y aporta estructura interpretable para análisis simbólico.

Entrenamiento prolongado del cabezal KAN (arquitectura $512 \rightarrow 32 \rightarrow 2$, grid mayor) con backbone **congelado**.

- Mejor **AUC en validación: 0,7198** (meseta alrededor de épocas 24–32).
- **Test con umbral 0,5:** AUC = **0,693**, acc = **0,647**, recall MALIGNANT = **0,747**.
- Con umbral ajustado por recall objetivo: recall MALIGNANT $\approx$ **0,810**; precisión $\approx$ **0,540** (misma AUC). Modelo guardado para interpretabilidad en el notebook 07.

Modelo híbrido ResNet-18 + KAN en el mismo grafo; fases con actualización opcional de rejilla de splines.

- Entrenamiento largo puede requerir desactivar o limitar `update_grid_from_samples` según versión de PyKAN/GPU.
- Paso opcional **LBFGS** sobre la KAN (ejemplo en salida): tras 5 pasos, val AUC $\approx$ **0,645** (no siempre mejora respecto al mejor Adam).
- **Test con checkpoint híbrido cargado** (salida registrada): acc **0,578**, AUC **0,636**, recall MALIGNANT **0,638**, F_1 mal. **0,555**.

Carga del cabezal KAN entrenado en 05; análisis cualitativo:

- `plot()`, importancia de aristas, `suggest_symbolic / auto_symbolic` sobre splines.
- Métricas de verificación al cargar pesos: AUC en test coherente con **~0,69** (mismo orden que el cabezal entrenado).
- Objetivo: traducir patrones en el espacio de embeddings a funciones explícitas y poda de aristas poco relevantes.

Híbrido end-to-end + herramientas de explicación en imagen y latente.

- **Val / Test** (ejemplo de salida): AUC val $\approx$ **0,680**, AUC test $\approx$ **0,635**; informe de clasificación por clase en test.
- **Sproud** (selección de aristas): del orden de **6 dimensiones latentes** únicas entre las aristas más influyentes.
- Integración con Captum (Integrated Gradients) sobre la entrada imagen; tabla simbólica + `symbolic_formula()` cuando la versión de PyKAN lo permite.

Tabla guía: lectura de matriz de confusión

Ejemplo (02_resnet18_tuned.ipynb, test): $\begin{pmatrix} 164 & 84 \\ 67 & 107 \end{pmatrix}$

Celda	Interpretación
164 (arriba izq.)	BENIGN bien clasificados (TN)
84 (arriba der.)	BENIGN etiquetados como MALIGNANT (FP)
67 (abajo izq.)	MALIGNANT perdidos como BENIGN (FN)
107 (abajo der.)	MALIGNANT detectados correctamente (TP)

Mensaje clave: en salud, minimizar FN (67) suele ser prioritario; por eso el ajuste de umbral busca subir recall de MALIGNANT.

Extracción de vectores 512-D con el ResNet afinado y splits alineados al manifiesto.

- Conteos típicos: Train **2315**, Val **549**, Test **422**.
- Embeddings y normalización guardados para entrenar la KAN fuera del bucle CNN (estabilidad en GPU).

KAN sobre embeddings con **PCA al 95 %** de varianza en train (ejemplo: **252** componentes en lugar de 512).

- Curva de entrenamiento: AUC en val sube hasta $\sim$ **0,679** alrededor de la época 34.
- **Test** (umbral elegido por recall en val): AUC **0,640**, acc **0,552**, recall MALIGNANT **0,833**, umbral $\approx$ **0,357**.
- Artefactos: `pca_payload.pt`, cargas interpretables en CSV.

Pipeline Sproud $\rightarrow$ ajuste simbólico arista a arista $\rightarrow$ `symbolic_formula()`.

- Salidas típicas: `sproud_edges.csv`, `symbolic_formula.txt`, gráficas de la red/fórmula si `matplotlib` está disponible.
- Validaciones para evitar ajustes triviales (`best_symbolic` vacío, $R^2 = 0$); posible fallback con `auto_symbolic`.
- Complementa el flujo principal: explicación compacta en el espacio PCA + KAN del experimento paralelo.

Objetivo de Prueba_red_KAN

Proyecto pequeño, aislado y didáctico para validar el flujo de entrenamiento con KAN sin el costo de mamografías:

- Dataset: **MNIST** (10 clases), ejecución en CPU.
- Paso 1 (`1_cnn_extractor.ipynb`): CNN simple para generar embeddings de 16 dimensiones.
- Paso 2 (`2_kan_explainer.ipynb`): entrenamiento KAN sobre embeddings, poda y reemplazo simbólico.
- Resultado: sandbox rápido para entender *cómo entrenar + interpretar* una KAN.

Pipeline técnico del mini-proyecto

Etapa	Qué se hace
1. CNN extractor	Conv(1→16) + Conv(16→32) + MLP; se entrena 1 época sobre MNIST.
2. Embeddings	Se exportan <code>train_embeddings.pt</code> y <code>test_embeddings.pt</code> (16-D).
3. KAN	Se entrena una KAN sobre embeddings durante 20 épocas.
4. Poda	<code>model.prune()</code> elimina conexiones poco activas.
5. Simbólico	<code>auto_symbolic(lib=[...])</code> + <code>symbolic_formula()</code> con variables Sympy.

Interpretación: replica en pequeño la idea central del proyecto principal: extractor profundo → modelo KAN → ecuaciones interpretables.

Resultados de entrenamiento KAN en MNIST

Hito	Valor	Lectura
Loss época 1	0.6329	Arranque con error alto
Loss época 5	0.1009	Mejora rápida del ajuste
Loss época 10	0.0919	Comienza estabilización
Loss época 20	0.0892	Convergencia práctica
Test Accuracy	96.98 %	Buen desempeño con embeddings compactos

Cómo leer esta tabla:

- La caída fuerte de loss en primeras épocas indica que la KAN aprende rápido sobre embeddings ya informativos.
- Accuracy cercana al 97 % confirma que el flujo extractor+KAN funciona de forma robusta.

Evidencia extraída de `2_kan_explainer.ipynb`:

- Tras `auto_symbolic`, muchas aristas se ajustan con función x y otras se anulan (0); se reportan varios R^2 altos (ej. 0.94–0.95 en algunas conexiones).
- Se obtiene fórmula simbólica explícita por clase (ejemplo mostrado en notebook para clase 0 y clase 1), con combinación lineal de $E_1, \dots, E_{16}$ y un término cuadrático.
- La poda reduce complejidad antes del paso simbólico, mejorando legibilidad de ecuaciones.

Conclusión práctica: este mini-proyecto valida que la estrategia usada en mamografías (embeddings + KAN + simbólico) es técnicamente viable y explicable.

Fórmulas simbólicas al final de 2_kan_explainer

En el notebook se extraen fórmulas explícitas para clases (ej. clase 0 y clase 1) usando:

- `model.auto_symbolic(lib=[...])`
- `formulas = model.symbolic_formula(var=mis_variables)[0]`

Ejemplo reportado para clase 0 (abreviado):

$$s_0(E) = \sum_{i=1}^{16} \alpha_i E_i + 0,0127(-0,8032 E_9 - 6,3721)^2 - 0,5827$$

y para clase 1:

$$s_1(E) = \sum_{i=1}^{16} \beta_i E_i - 0,0110(-0,8032 E_9 - 6,3721)^2 - 8,0311$$

donde $E_1, \dots, E_{16}$ son los embeddings de la CNN.

De fórmula simbólica a probabilidad de clase

La interpretación operativa es:

- 1 Se toma un embedding $E \in \mathbb{R}^{16}$ de una imagen.
- 2 Se evalúan las fórmulas simbólicas de cada clase para obtener **logits** $s_k(E)$.
- 3 Se convierten a probabilidades con softmax:

$$p(y = k \mid E) = \frac{e^{s_k(E)}}{\sum_j e^{s_j(E)}}$$

Argumento clave para la presentación:

- Para este caso sí se puede usar una fórmula explícita con valores de embeddings.
- Esa fórmula produce un score; al normalizarlo con softmax se obtiene probabilidad alta para la clase más compatible.
- Por eso la KAN no solo clasifica: también deja una regla matemática interpretable y evaluable fuera del modelo.

- Del CSV masivo al manifiesto: **422** casos de test consistentes en todo el pipeline.
- CNN afinada: test AUC $\approx$ **0,70**; umbral operativo prioriza **recall de malignos** (~ 80 %).
- KAN sobre embeddings y cabezal dedicado: AUC test del orden **0,69–0,70**; interpretabilidad vía Sproud/simbólico e IG en 07/08.

- README.md, Prueba_embeddings/README.md.
- scripts/paths.py, scripts/notebook_utils.py.
- Dataset CBIS-DDSM (TCIA / licencia de uso).